Multi-Agent AI Customer Support Assistant using RAG and LLMs

Capstone Project Report

1. Abstract

This project implements a full-stack web-based AI customer support platform in which a central orchestrator detects customer intent and routes requests to one or more specialized agents. Billing, Technical, Product, Complaint and FAQ agents are grounded by Retrieval-Augmented Generation. SentenceTransformer embeddings and FAISS provide semantic retrieval, while an OpenAI-compatible LLM produces context-aware answers. Authenticated conversation memory and automatic escalation tickets complete the support workflow.

2. Problem Statement

A single generic chatbot may struggle across billing, technical, product and complaint domains. The implemented architecture separates responsibilities, allows multi-agent routing for composite requests, and retrieves company-specific information before generation.

3. Architecture

React Web Chat -> FastAPI -> JWT Authentication and Conversation Memory -> Intent Detection -> Agent Router -> Specialized Agent(s) -> RAG Retriever -> FAISS -> Company PDF Knowledge Base -> LLM -> Response Aggregator -> Final Response -> Optional Human Escalation.

4. Frontend

The React interface provides registration/login, a chat window, recent conversations, typing feedback, selected-agent badges, response latency, retrieved-source disclosure and escalation status.

5. Backend and Database

FastAPI exposes authentication, chat, conversation, analytics and RAG endpoints. SQLAlchemy models Users, Conversations, Messages and Tickets. Local demonstration can use SQLite; PostgreSQL is supported through DATABASE_URL for deployment.

6. Multi-Agent Architecture

Intent detection can select one or multiple agents. The composite query 'I paid yesterday but Premium is still locked' routes to Billing and Technical agents. Each specialist receives recent memory and retrieved company context. Multiple outputs are combined by a response aggregator.

7. RAG

PyPDF extracts the company documents. Text is split into overlapping chunks and encoded with sentence-transformers/all-MiniLM-L6-v2. Normalized embeddings are stored in a FAISS inner-product index. Incoming questions are embedded and the most similar chunks are returned with source file and page metadata.

8. LLM Integration

The OpenAI Python SDK is used as an OpenAI-compatible client. Provider base URL and model are configurable, allowing OpenAI or compatible services such as Groq. Agent prompts require grounding in retrieved context and prohibit fabricated policies or account facts.

9. Conversation Memory and Escalation

Messages are stored with role, timestamp, session/conversation association, agent metadata and response latency. Recent messages are included in agent context. Escalation occurs when retrieval confidence is low, an agent recommends human review, or a negative complaint remains unresolved.

10. Testing and Evaluation

Unit tests cover billing, technical, complaint and multi-agent routing and RAG chunking. An evaluation script compares predicted routes with expected routes and records routing latency. End-to-end retrieval can be evaluated by asking policy-specific questions and checking the displayed source PDF.

11. Security

Passwords use PBKDF2-HMAC-SHA256 with random salts. JWT protects authenticated endpoints. Conversation queries are scoped to the authenticated user. CORS is configurable. LLM prompts require uncertainty to be surfaced rather than fabricated.

12. Deployment

The React frontend can be deployed to Vercel. FastAPI can be deployed to Render or Railway. PostgreSQL can be used as the persistent production database. Required environment variables include DATABASE_URL, JWT_SECRET, LLM_API_KEY, LLM_BASE_URL, LLM_MODEL and CORS_ORIGINS.

13. Demonstration

The final video should show each specialist agent, the multi-agent Billing plus Technical scenario, RAG source disclosure, conversation memory, automatic escalation, unit tests and routing evaluation.

14. Conclusion

The completed capstone demonstrates multi-agent orchestration, RAG, embeddings, a vector database, LLM integration, REST APIs, full-stack development, database design, conversation memory, testing and deployment-ready architecture.